

TASTE HUT

Multi-Tenant SaaS Platform

System Requirements & Technical Design Document — v2.0

github.com/MueezulRehman/Restaurants-System

Version 2.0 | July 2026

1. Executive Summary

TasteHut is a cloud-based, multi-tenant SaaS platform that allows a single Super Admin to onboard and manage multiple independent businesses — restaurants, retail stores, cafes, or any point-of-sale business. Each business gets its own fully isolated environment: its own staff, inventory, POS, sales records, expenses, invoices, feedback, and customer-facing storefront.

The system supports three primary roles:

- **Super Admin** — owns and operates the SaaS platform. Onboards businesses, sets their type, controls which modules they can access, manages subscription plans, and reviews all feedback.
- **Business Manager** — manages a single business. Has access only to the modules granted by Super Admin: menu/inventory, POS, orders, sales, expenses, staff, attendance, salary, reports, and customer feedback.
- **Customer** — browses a restaurant's public storefront, adds items to cart, places orders, tracks delivery, and submits feedback to the manager.

2. Business Types Supported

When the Super Admin registers a new business, they select the business type. The type determines which modules and sidebar items the Manager sees, making the same codebase serve multiple verticals.

Business Type	Modules Enabled	Customer Storefront
Restaurant	POS, Menu, Categories, Variants, Deals, Orders, Cashbook, Expenses, Staff, HR, Reports, Feedback	Yes — full menu + ordering
Retail / Shop	POS, Inventory, Categories, Variants, Cashbook, Expenses, Staff, HR, Reports, Feedback	No (admin POS only)
Cafe / Bakery	POS, Menu, Categories, Variants, Deals, Orders, Cashbook, Expenses, Staff, HR, Reports, Feedback	Optional
General Business	POS, Inventory, Variants, Cashbook, Expenses, Staff, HR, Reports, Feedback	No

Note: Future business types can be added without changing the core codebase — only the `enabled_modules` JSON column on the `restaurants` table needs to be updated.

3. Role Separation

3.1 Super Admin

Super Admin is the platform owner. After login they land on the Businesses dashboard. Their sidebar contains only platform-level tools:

- **Businesses** — add, edit, activate, deactivate, assign plan
- **Business Type selector** — pick type and enable/disable modules per business
- **Subscription & Plans** — view active subscriptions, overdue payments, trial expirations
- **Feedback Centre** — read all manager suggestions and customer complaints across all businesses

Note: Super Admin has ZERO access to any business's data (menu, orders, cashbook, staff). Visiting `/admin/dashboard` or `/admin/orders` as super admin returns HTTP 403.

3.2 Business Manager

Each Manager is linked to exactly one business via `restaurant_id`. Their sidebar is dynamically built from the modules enabled by Super Admin. They can only see and edit their own business's data — the `BelongsToRestaurant` global Eloquent scope enforces this at the database layer.

Possible sidebar sections (shown only if module is enabled for this business):

- **Dashboard** — KPI cards, low-stock alerts, recent activity
- **POS / Billing** — scan or search item (by barcode, SKU, or name) → select variant → add to cart → generate invoice
- **Orders** — list, view detail, update status (restaurant/cafe type only)
- **Menu / Inventory** — categories, items with variants, deals
- **Financial** — cashbook, expenses
- **Staff & HR** — staff list, attendance, salary
- **Reports** — sales, income/expense, staff — filterable by date range, month, 6 months, year, past years
- **Feedback** — view customer feedback, send suggestions to Super Admin

3.3 Customer

Customers access the public storefront at the restaurant's custom domain or subdomain (e.g. `myhut.tastehut.com` or `myhut.com`). They can:

- Browse menu by category
- Select item variants (size, colour, storage, etc.) before adding to cart
- Checkout with delivery or pickup
- Track order status via unique tracking token
- Submit feedback/rating to the restaurant manager

Note: Customers can only see their own orders. No customer can view another customer's order details, tracking page, or receipt — each order uses a unique unguessable tracking token.

4. Subscription Plans & Billing

TasteHut charges businesses a recurring subscription fee. Both monthly and annual billing cycles are supported. Annual billing offers a discount to incentivise long-term commitment.

4.1 Plan Tiers

Plan	Price / Month	Price / Year	Modules Included	Business Limit
Starter	Rs. 2,500	Rs. 25,000 (save 17%)	POS, Menu/Inventory, Categories, Basic Reports	1 business
Growth	Rs. 6,000	Rs. 60,000 (save 17%)	All Starter + Staff HR, Expenses, Cashbook, Invoices, Customer Storefront, Custom Subdomain, Feedback	1 business
Pro	Rs. 12,000	Rs. 1,20,000 (save 17%)	All Growth + Custom Domain, Advanced Reports (past years), Variants, Priority Support	1 business

Plan	Price / Month	Price / Year	Modules Included	Business Limit
Enterprise	Custom	Custom	All Pro + multiple locations, dedicated support, custom modules	Multiple

4.2 Billing Cycle

- **Monthly** — charged on the same date each month. Manager can cancel anytime; access continues until the end of the paid period.
- **Annual** — charged once per year at a ~17% discount vs monthly. Non-refundable after 7 days.
- **Trial** — every new business gets 14 days free on the Growth plan, no payment details required. After 14 days, they must select a plan or the account is frozen (data preserved, access blocked).

4.3 Auto-Deactivation Rules

- If payment fails or subscription expires, the business is automatically set to `is_active = false`
- Manager login still works but every page redirects to a "Subscription Expired" screen with a Pay Now button
- Customer storefront is taken offline (shows a "temporarily unavailable" page)
- All data is preserved — nothing is deleted on expiry
- Super Admin can manually reactivate or extend a grace period

4.4 Payment Gateways (Supported)

- **JazzCash** — primary gateway for Pakistan
- **EasyPaiza** — secondary gateway
- **Stripe** — for international businesses
- **Manual / Bank Transfer** — Super Admin marks payment as received manually (for Enterprise clients)

Note: Auto-billing via gateway webhook is required. Without it, Super Admin must chase payments manually which does not scale.

5. POS — Point of Sale

The POS module is the core selling tool for the Manager. It must support fast, keyboard and scanner-driven operation with no mouse clicks required for a standard sale.

5.1 Item Lookup — Four Methods

- **Barcode Scanner** — plug in any USB HID scanner; the POS page listens to rapid keystrokes followed by Enter and auto-adds the item to cart
- **Item Number / SKU** — type the SKU and press Enter
- **Item Name Search** — type partial name; a live-search dropdown shows matches instantly
- **Category Browse** — click a category tab to see the item grid, then click to add

5.1.1 Barcode Entry Workflow — Adding an Item (Implementation Spec)

There are exactly two ways a manager gets a barcode into an item's record. Both write to the same field, so either one is sufficient — the build must support both.

- **Way 1 — Type it manually.** On Menu Items → Add New Item, the form has a plain text "Barcode" field. The manager types or pastes the barcode number in (e.g. 6901443391356) and saves. No special formatting or validation beyond uniqueness-per-business is required.
- **Way 2 — Scan directly into the form (easiest).** The manager plugs in a USB barcode scanner, clicks inside the Barcode field on the Add Item form, then points the scanner at the product. The scanner fires the number into the field automatically — functionally identical to typing it, just instant and error-free. No click-to-add step or special handler is needed; the field just needs focus.

Full item setup flow (step by step):

- 1. Manager opens Add New Item
- 2. Clicks the Barcode field → scans the product → number fills in automatically (or types/pastes it manually)
- 3. Fills in the rest: name, category, price, cost price, stock quantity
- 4. If the item has variants (colour, size, etc.) → enables the Variants toggle and adds them
- 5. Uploads a photo (optional)
- 6. Saves

Then, at the POS, when that same product is scanned at the counter, the system looks up the barcode in the database and adds it to the cart instantly — this is the exact mechanism described in 5.1's "Barcode Scanner" method.

⚠ Note: A barcode scanner is just a keyboard in disguise — it types the number and presses Enter. No special software or driver is needed; any cheap USB HID scanner works out of the box. This means the "Barcode" input on both the Add Item form and the POS search box are ordinary text inputs — no scanner-specific SDK, browser permission, or hardware integration is required for USB scanning. (This is separate from optional camera-based scanning, which is an added convenience, not a requirement.)

5.2 Product Variants

When an item has variants (e.g. iPhone 16 with options for colour, storage size, or battery health), the manager selects the variant before the item is added to cart. Variants are configured per item in the Menu/Inventory section.

Variant examples:

- **Colour** — Black, White, Desert Titanium, Natural Titanium
- **Storage Size** — 128GB, 256GB, 512GB, 1TB
- **Battery Health** — 100%, 95–99%, 90–94%
- **Size** — S, M, L, XL (clothing)
- Each variant combination can have its own price and stock quantity

⚠ Note: When a variant item is added to cart, the cart line shows: Item Name + Variant (e.g. "iPhone 16 — Black / 256GB"), Qty, Unit Price, Line Total.

5.3 Cart & Billing

- Each cart line shows: item name, selected variant, qty (editable inline), unit price, line total
- Manager can adjust quantity, apply item-level or cart-level discount, or remove a line
- Cart summary: subtotal, discount, tax (configurable per business), grand total
- Payment modes: Cash, Card, Split (future)
- On "Complete Sale": order saved to DB, stock decremented per variant, invoice auto-generated

5.4 Invoice / Bill

- Auto-generated PDF with: business logo, name, address, bill number, date/time, itemised list with variants, totals, payment mode
- Bill numbers are sequential per business: INV-0001, INV-0002 ...
- Print directly from browser or download as PDF

6. Menu & Inventory Management

6.1 Categories

- CRUD: name, icon, sort order, active/inactive
- Unique per business — two businesses can both have a "Pizza" or "Phones" category

6.2 Items / Menu Items

- CRUD: name, SKU, barcode, category, description, image, base price, cost price, stock qty, unit
- Active / Inactive toggle — inactive items do not appear in POS or storefront
- Has Variants toggle — if enabled, variants are configured instead of a single price/stock

6.3 Variants

- Manager defines variant groups (e.g. "Colour", "Storage") and options within each group
- Each variant combination has its own SKU, price, cost price, and stock quantity
- Variant matrix is auto-generated from the options (e.g. 3 colours × 4 sizes = 12 variants)

6.4 Deals / Combos

- Bundle multiple items or variants at a fixed deal price
- Validity: start and end date
- Shown in POS and customer storefront

7. Order Privacy & Handling

7.1 Customer Order Privacy

Each order is assigned a unique, cryptographically random tracking token at creation time. Customers can only access an order via its token URL (e.g. /track/a3f9c2b1). There is no order listing page for customers — they cannot browse or search other orders.

- Token is generated with Laravel's Str::uuid() or random_bytes(16) — not guessable
- Customer tracking page shows only: order status, items ordered, total — no other customer's name, phone, or address is ever visible
- No authentication required for tracking — just the token URL
- If someone tries to guess a token and it doesn't exist, they receive a 404 — not a "wrong token" message

7.2 Manager Order Handling

- **New Order** — appears in Manager's order list immediately; optional browser notification
- **Status Flow** — Order Received → Confirmed → Preparing → Out for Delivery → Delivered (or Cancelled)
- **Order Detail** — Manager sees full detail: customer name, phone, address, items, variants, total, notes
- **Cross-restaurant isolation** — Manager from Restaurant A can never see Restaurant B's orders, even by guessing an order ID in the URL (BelongsToRestaurant scope returns 404)

8. Financial Management

8.1 Cashbook

- Record any credit (income) or debit (expense) entry manually
- Each POS sale auto-creates a credit entry
- Running balance shown at all times
- Filter by date range

8.2 Expenses

- Categorised expense entries (rent, utilities, salaries, supplies...)
- Attach receipt image
- Monthly summary shown on dashboard

9. Reports

All reports are available to the Manager and exportable as PDF or Excel. The date filter supports multiple preset ranges and custom range selection.

9.1 Date Filter Options

Filter	What It Shows
Today	Sales, expenses, orders for today
This Week	Current Mon–Sun
This Month	Current calendar month
Last 6 Months	Rolling 6-month window
This Year	Jan 1 – Dec 31 of current year
Past Years	Manager selects any previous year from a dropdown (all historical data preserved)
Custom Range	Manager picks start date and end date manually

9.2 Report Types

- **Sales Report** — total sales, number of orders, average order value, top-selling items — by selected period
- **Income vs Expense Report** — side-by-side comparison of cashbook credits vs debits, net profit/loss

- **Item Performance** — units sold, revenue, and profit margin per item/variant
- **Staff Attendance Summary** — present/absent/late count per staff member for the selected period
- **Salary Report** — monthly salary payouts, deductions, paid/unpaid status

⚠ Note: Past years' data is never deleted when a business renews or changes plan. All historical records remain accessible.

10. Feedback & Suggestions System

The platform has a two-layer feedback flow: customers give feedback to managers, and managers give suggestions/feedback to the Super Admin.

10.1 Customer → Manager Feedback

- After an order is delivered, customer can submit a rating (1–5 stars) and a written comment
- Feedback is tied to the specific order and restaurant
- Manager sees all feedback in a Feedback section of their sidebar
- Manager can mark feedback as "Reviewed" or "Resolved"
- No customer can see other customers' feedback

10.2 Manager → Super Admin Suggestions

- Manager has a "Send Suggestion / Report Issue" form in their sidebar
- They can submit: feature requests, bug reports, complaints, or general feedback
- Each submission is tagged with the business name and manager name automatically
- Super Admin sees all submissions in a Feedback Centre on their dashboard
- Super Admin can reply, mark as resolved, or tag for development

10.3 Feedback Visibility Rules

Who Submits	Who Receives	Who Can See It
Customer	Restaurant Manager	Manager of that restaurant only; Super Admin can also view all
Manager	Super Admin	Super Admin only; the submitting manager can see their own submissions and replies
Super Admin	Manager (reply)	The specific manager who submitted the original suggestion

11. Dashboard & KPIs

11.1 Manager Dashboard

KPI Card	Data Source
Orders Today	orders table, today, this restaurant_id
Revenue Today	orders SUM(total), today, this restaurant_id

KPI Card	Data Source
Pending Orders	orders WHERE status = pending
Expenses Today	expenses SUM(amount), today
Low Stock Items	menu_item_variants WHERE stock_qty < threshold
Monthly Net Profit	cashbook credits – debits, current month
New Customer Feedback	feedbacks WHERE reviewed = false

Below the KPI cards: Recent Orders table + Weekly Revenue chart + Low Stock alerts.

11.2 Super Admin Dashboard

KPI Card	Data Source
Total Active Businesses	restaurants WHERE is_active = true
Trials Expiring This Week	restaurants WHERE trial_ends_at <= now + 7 days
Overdue Subscriptions	subscriptions WHERE status = overdue
Revenue This Month	subscriptions SUM(amount_paid), this month
New Feedback / Suggestions	manager_feedbacks WHERE status = new

12. Customer Storefront (Restaurant & Cafe Type)

- Accessible at the restaurant's custom domain or subdomain
- Menu browsing by category with item images and prices
- Variant selector on item detail — customer picks colour/size/storage before adding to cart
- Cart sidebar — add/remove items, adjust qty, view total
- Checkout: name, phone, address, order type (Delivery / Pickup), notes
- Order tracking via unique token URL — customer bookmarks it or uses the lookup form
- Post-delivery: feedback form with star rating and comment

⚠ Note: Customer storefront is only accessible when the business subscription is active. If subscription expires, storefront shows a "temporarily unavailable" page.

13. Multi-Tenancy & Data Isolation

Every tenant model carries a `restaurant_id` and uses a global Eloquent scope (BelongsToRestaurant trait) that automatically filters every query to the logged-in user's `restaurant_id`. Key guarantees:

- A manager from Business A cannot read, edit, or delete Business B's data — even by guessing a record ID in the URL (the row won't be found; Laravel returns 404)
- New records automatically inherit the creator's `restaurant_id`
- Super Admin is exempt from this scope for platform-level queries
- Customer orders are isolated by tracking token — no enumeration possible

13.1 Future-Proofing: Custom DB Structure Per Business

Some enterprise clients may eventually require a completely separate database or schema (e.g. a large retail chain that wants their data on their own server). The current architecture already supports this via the following approach:

- **Step 1** — Add a `db_connection` column to the `restaurants` table (default: `null` = shared DB)
- **Step 2** — Store the custom DB credentials (host, database, username, password) encrypted in that column
- **Step 3** — In the `BelongsToRestaurant` trait's `bootBelongsToRestaurant()` method, before applying the scope, switch the DB connection: `DB::purge()` → `Config::set('database.connections.tenant', ...)` → `DB::reconnect('tenant')`
- **Step 4** — The rest of the application works identically — no controller or model changes needed

13.2 Migrating an Existing Business to a New System

If a business needs to move their historical data to a new or upgraded system, the safe migration process is:

1. Export all records for that `restaurant_id` using `php artisan db:export --restaurant=X` (custom artisan command to be built)
2. The export produces a JSON or SQL dump scoped to that business only — no other business data included
3. Import into the new system/database using the matching import command
4. Validate record counts and spot-check key records (orders, invoices, cashbook)
5. Update the `db_connection` column if the business is moving to their own database

⚠ Note: No data is ever deleted during migration. The original records stay in the shared DB until the business explicitly requests deletion (right to erasure).

14. Technical Stack

Layer	Technology
Backend Framework	Laravel 11 (PHP 8.2+)
Frontend — Admin	Blade + Tailwind CSS
Frontend — Customer	Blade + Tailwind CSS (TasteHut storefront design)
Database	MySQL 8 / MariaDB 10.5+ (shared, with optional per-business DB)
Authentication	Laravel Auth — session-based, separate admin guard
PDF Generation	Barryvdh Laravel-DomPDF (invoices + reports)
Excel Export	Maatwebsite Laravel-Excel (report exports)
Barcode Scanning	Native USB HID keyboard-emulation — no extra library needed
File Storage	Laravel Storage (local disk or AWS S3)
Payment Gateway	JazzCash, EasyPaiza, Stripe (via webhook)
Queue / Jobs	Laravel Queue — invoice generation, notifications
Notifications	WhatsApp (via API), Email (Laravel Mail + SMTP)
Hosting	Any VPS with PHP 8.2+ and MySQL (DigitalOcean, AWS, shared cPanel)

15. Database Schema (Current Implementation)

This section reflects the actual tables in the codebase today, generated directly from the migration files — not the target/planned schema. Tables described in the Development Roadmap (Section 18) as not-yet-built (subscriptions, invoices, item variants, feedback tables) are intentionally not listed here; see the Roadmap for what's planned.

15.1 Platform & Accounts

restaurants — one row per tenant business (restaurant or shop)

Column	Type & Notes
id, name, slug (unique)	identity + public URL slug
email, phone, address	nullable contact info
custom_domain (unique), domain (unique)	both nullable — custom domain support
plan	string, default 'basic' — informal today, formal tiers are Roadmap Phase 4
business_type	string, default 'restaurant' — 'restaurant' or 'shop'
status	enum: trial active suspended cancelled
logo_path, theme, trial_ends_at	logo image path; theme stored as text; nullable

restaurant_domains — extra mapped domains per restaurant (beyond the single domain/custom_domain columns above)

Column	Type & Notes
restaurant_id	FK → restaurants, cascade delete
domain (unique), is_primary	one restaurant can have several domains
status	enum: pending verified blocked

users — staff accounts — super admin, restaurant admin/manager, cashier, kitchen, rider

Column	Type & Notes
restaurant_id	FK → restaurants, nullable (null for super_admin), null on delete
name, phone (unique), email (unique, nullable)	phone is the login identifier, not email
role	enum: super_admin admin cashier kitchen rider staff manager
password	bcrypt hash
monthly_salary, is_active, joined_at	monthly_salary nullable

customers — platform-wide customer accounts (NOT restaurant-scoped — one account orders across any restaurant on the platform)

Column	Type & Notes
name, phone (unique), email (unique, nullable)	phone is the login identifier
password	bcrypt hash, own auth guard ('customer'), separate from staff 'web' guard
default_address	nullable, prefills checkout

15.2 Restaurant Menu & Ordering

categories — menu categories, also reused as Product categories for shop-type businesses

Column	Type & Notes
restaurant_id	FK → restaurants, nullable, null on delete
name, slug (unique), icon	slug currently unique globally, not per-restaurant — see note below
sort_order, is_active	

⚠ **Note:** categories.slug has a global unique constraint, not a per-restaurant one — two different restaurants cannot both have a category with the same slug today. Worth revisiting as a composite unique(restaurant_id, slug) if this causes collisions in practice.

menu_items — food items for restaurant-type businesses

Column	Type & Notes
restaurant_id, category_id	both FK, restaurant_id null on delete
name, description	
price	nullable — used directly when the item has no sizes
has_sizes, allows_toppings	booleans — gate whether size/topping pickers show
image, is_available, sort_order	

menu_item_sizes — e.g. Small/Medium/Large pricing for a has_sizes item

Column	Type & Notes
menu_item_id	FK → menu_items, cascade delete
size_label, price, sort_order	

toppings — extra toppings/sauces addable to an item

Column	Type & Notes
name, price	
type	enum: topping sauce extra

⚠ **Note:** toppings has no restaurant_id — it is currently a single global list shared by every restaurant on the platform, not tenant-scoped. Likely fine while there's one restaurant using it; worth adding restaurant_id before a second restaurant needs its own distinct topping list.

deals — combo/bundle pricing

Column	Type & Notes
restaurant_id	FK, nullable, null on delete
name, deal_number, price, description, image, is_active	

15.3 Orders

orders — one row per placed order (guest or logged-in customer)

Column	Type & Notes
restaurant_id	FK, nullable, null on delete
customer_id	FK → customers, nullable — set when a logged-in customer checks out, null for guests
order_number (unique)	e.g. TH-20260710-0004
tracking_token (unique)	UUID-style, used in the no-login order-tracking URL
order_type	enum: dine_in takeaway delivery online

Column	Type & Notes
status	enum: pending confirmed preparing ready out_for_delivery delivered cancelled
customer_name, customer_phone (indexed), address	address required for delivery orders
subtotal, delivery_fee, total	
payment_method	enum: cash online
notes, estimated_minutes	
confirmed_at, ready_at, delivered_at	nullable timestamps for each stage

order_items — line items within an order

Column	Type & Notes
order_id	FK, cascade delete
item_type	enum: menu_item deal
menu_item_id, deal_id	both FK, nullable, null on delete
item_name	snapshot — survives menu changes after the order is placed
size_label, quantity, unit_price, total_price, special_request	

order_item_toppings — toppings picked for a specific order item

Column	Type & Notes
order_item_id, topping_id	both FK, cascade delete
topping_name, price	topping_name is a snapshot

deliveries — delivery assignment/status per order

Column	Type & Notes
order_id	FK, cascade delete
rider_id	FK → users, nullable, null on delete
status	enum: pending assigned on_the_way delivered
assigned_at, delivered_at, delivery_notes	

15.4 Financial & HR (Restaurant-Type)

cashbook — manual + auto-generated cash in/out entries

Column	Type & Notes
restaurant_id	FK, nullable, null on delete
type	enum: in out
amount, description, date	
source	enum: order manual expense
order_id	FK, nullable — links auto-entries back to the order
created_by	FK → users, required

expenses — restaurant expense records

Column	Type & Notes
restaurant_id	FK, nullable, null on delete
category	enum: rent salary supplies utilities marketing other
amount, description, date, receipt_image	
created_by	FK → users, required

attendance — daily staff attendance

Column	Type & Notes
restaurant_id, user_id	both FK, restaurant_id nullable
date, status	status enum: present absent half_day leave
check_in, check_out, notes	
unique constraint	(user_id, date) — one record per staff per day

salaries — monthly salary payment records

Column	Type & Notes
restaurant_id, user_id	both FK, restaurant_id nullable
month	stored as the 1st of the month, e.g. 2026-06-01
amount, deductions, net_paid, paid_at	
paid_by	FK → users, nullable, null on delete
unique constraint	(user_id, month)

15.5 Retail / Shop POS (Business Type: shop)

products — sellable items for a shop-type business (e.g. garments)

Column	Type & Notes
restaurant_id	FK, cascade delete
category_id	FK → categories, nullable, null on delete — reuses the same Category model
name, code, size, color	code = barcode/SKU. unique constraint on (restaurant_id, code)
price, cost_price	cost_price nullable
stock_quantity, low_stock_threshold	defaults 0 and 5
is_active	

sales — one row per bill/invoice generated at the POS register

Column	Type & Notes
restaurant_id	FK, cascade delete
user_id	FK → users, nullable, null on delete — the cashier
invoice_number	e.g. INV-20260710-0004, per-restaurant daily sequence
customer_name, customer_phone	both nullable — walk-in sales have neither
subtotal, discount_amount, tax_amount, total	
payment_method	default 'cash'

sale_items — line items within a bill

Column	Type & Notes
sale_id	FK, cascade delete
product_id	FK → products, nullable, null on delete — bill still shows what was sold via the snapshot columns even if the product is later deleted
product_name, code	snapshots at time of sale
unit_price, quantity, line_total	

Framework tables not listed above (sessions, jobs, cache) are standard Laravel infrastructure tables, not application data.

16. Roles, Permissions & Access Control (Current Implementation)

Access control today is enforced by a combination of three mechanisms — not by named route gates (e.g. a 'can:super-admin-only' policy). Documenting the real mechanism matters because it changes where a future developer should look when debugging an access issue.

16.1 The Three Enforcement Layers

Layer 1 — Auth guards — two separate Laravel auth guards

Column	Type & Notes
web guard	staff accounts (super_admin, admin/manager, cashier, kitchen, rider) — the users table
customer guard	customer accounts — the customers table, entirely separate session/login

Layer 2 — Controller-level role checks — PHP checks inside controllers, not route middleware

Column	Type & Notes
User::isSuperAdmin()	true only for role === 'super_admin'
User::isRestaurantManager()	true for role in ['admin', 'manager']
abort_unless(...)	used in e.g. ManagerDashboardController to 403 anyone who isn't staff for that restaurant

Layer 3 — BelongsToRestaurant trait — a global Eloquent scope on every tenant-owned model

Column	Type & Notes
Auto-filter	queries automatically add WHERE restaurant_id = X for the current user's restaurant
Auto-fill	new records automatically get restaurant_id set on create
Super admin exception	unscoped (sees everything) unless currently impersonating a restaurant via the Tenancy helper
Applies to	Category, Menuitem, Deal, Order, Cashbook, Expense, Attendance, Salary, Product, Sale

⚠ Recommendation: route-level role middleware (e.g. a simple EnsureRole::class middleware checking the role before the controller even runs) would add a defense-in-depth layer on top of Layer 2's controller checks. Today, a mistake in a single controller's abort_unless() call is the only thing standing between a logged-in user and a route meant for a different role. Not urgent while the team is one developer, but worth adding before onboarding other developers.

16.2 Permissions Matrix

Capability	Super Admin	Restaurant Admin/Manager	Customer
Create/edit/suspend any restaurant	Yes	No	No

Capability	Super Admin	Restaurant Admin/Manager	Customer
Enter & manage a specific restaurant (impersonate)	Yes, via Tenancy	N/A — already scoped to own restaurant	No
View own restaurant's menu/products/orders/sales	Only while impersonating	Yes, own restaurant only	No
View another restaurant's data	Only while impersonating that one	No — blocked by BelongsToRestaurant	No
Place an order / generate a POS bill	Only while impersonating (POS)	Yes (POS, for shop type)	Order: yes. POS: no
View own order history	N/A	N/A	Yes, if logged in
View own past orders as guest	N/A	N/A	Via tracking token link only, no login needed
Manage staff / attendance / salary	Only while impersonating	Yes, own restaurant only	No

16.3 Route Groups (Actual)

Route Prefix	Middleware (Actual)	Effective Access
/admin/login, /admin/logout	guest / auth (web guard)	super_admin only (enforced in AuthController)
/manager/login, /manager/logout	guest / auth (web guard)	admin, manager roles (enforced in ManagerAuthController)
/admin/dashboard	auth	super_admin (platform view or impersonating-restaurant view); non-super-admin redirected to /manager/dashboard
/manager/dashboard	auth	admin/manager, or an impersonating super_admin
/admin/categories, /menu-items, /deals, /products, /pos, /sales, /cashbook, /expenses, /staff, /attendance, /salary, /orders	auth	Shared by super_admin (while impersonating) and restaurant admin/manager — isolation enforced by the BelongsToRestaurant trait, not route middleware
/admin/restaurants*	auth	super_admin only (controller-level check)
/register, /login, /logout, /account	guest:customer / auth:customer	customer guard only, separate from staff
/ (storefront), /checkout, /track/{token}	ResolveRestaurant (public)	anyone — guests and logged-in customers alike

17. Security Checklist

Grounded in what's actually in the codebase today, not a generic enterprise checklist. "Implemented" items were confirmed by reading the actual code; "Recommended" items are gaps worth closing, roughly in priority order.

Item	Status	Notes
Password hashing	✓ Implemented	bcrypt via Hash::make(), both staff and customer accounts
CSRF protection	✓ Implemented	Laravel default — @csrf on every form found in the codebase

Item	Status	Notes
Tenant data isolation	✓ Implemented	BelongsToRestaurant global scope + explicit ownership checks (e.g. OrderController)
Separate customer/staff sessions	✓ Implemented	distinct auth guards, can't cross-authenticate
Mass assignment protection	✓ Implemented	\$fillable allowlists on every model checked
SQL injection protection	✓ Implemented	Eloquent/query builder parameter binding throughout — no raw concatenated SQL found
XSS protection	✓ Implemented	Blade {{ }} auto-escapes by default; no {!! !!} raw-output misuse found in reviewed views
.env not committed	✓ Implemented	confirmed present in .gitignore
Route-level role middleware	⚠ Recommended	currently controller-level only (see 16.1) — add as defense-in-depth
Login rate limiting	⚠ Recommended	Laravel's built-in throttle middleware isn't yet applied to /admin/login, /manager/login, /login
HTTPS enforcement in production	⚠ Verify	confirm APP_URL uses https:// and consider forcing scheme + HSTS header in production
Secure session cookies	⚠ Verify	confirm SESSION_SECURE_COOKIE=true and SESSION_ENCRYPT are set in the production .env
File upload validation	⚠ Verify	confirm logo/receipt image uploads validate mime type and size server-side, not just via the HTML accept attribute
Database backups	⚠ Recommended	no backup strategy in place yet — worth automating before real customer data accumulates
Dependency updates	⚠ Ongoing	run composer audit / npm audit periodically as the project grows

18. Development Roadmap

⚠ **Note:** All phases below are part of the current build. Nothing is deferred to "future".

Phase	Deliverables	Status
Phase 1 — Core & Auth	Multi-tenancy, data isolation (BelongsToRestaurant), role separation (super admin vs manager vs customer), sidebar role-aware nav, route gates	✓ Done
Phase 2 — POS & Variants	POS screen, barcode scanner input, item search (SKU/name), product variants (colour/size/storage etc.), cart, invoice PDF generation	☐ In Progress
Phase 3 — Business Types & Modules	Business type selector on registration, dynamic module enable/disable per business, sidebar built from enabled modules	☐ Next
Phase 4 — Subscription & Billing	Plan tiers (Starter/Growth/Pro/Enterprise), monthly & annual billing cycles, 14-day trial, auto-deactivation on expiry, JazzCash + EasyPaisa + Stripe integration	☐ Next
Phase 5 — Reports	Sales, income/expense, item performance, staff summary — with Today / Week / Month / 6 Months / Year / Past Years / Custom Range filters — PDF + Excel export	☐ Planned

Phase	Deliverables	Status
Phase 6 — Feedback System	Customer → Manager feedback with star rating, Manager → Super Admin suggestions, Super Admin Feedback Centre with reply and status tracking	☐ Planned
Phase 7 — Notifications	New order browser notification, WhatsApp invoice delivery, order status SMS/WhatsApp to customer, low-stock email alert to manager	☐ Planned
Phase 8 — Stock Management	Per-variant stock tracking, stock adjustment history, low-stock threshold alerts on dashboard	☐ Planned
Phase 9 — Custom DB & Migration Tools	Per-business database connection support, artisan export/import commands for data migration, right-to-erasure delete command	☐ Planned

19. Setup & Installation

Requirements

- PHP 8.2+, Composer, MySQL 8+, Node.js 18+

Steps

1. Clone:

```
git clone https://github.com/MueezulRehman/Restaurants-System.git
```

2. Install:

```
composer install && npm install && npm run build
```

3. Configure:

```
cp .env.example .env && php artisan key:generate
```

4. Migrate & Seed:

```
php artisan migrate --seed
```

5. Serve:

```
php artisan serve
```

Default Super Admin login after seed:

Email: superadmin@tastehut.com | Password: password